



Instituto Tecnológico Superior de Lerdo

“Desarrollo de Aplicaciones para Dispositivos Móviles”

Unidad 2

Actividad de evaluación ABPj C2 - Análisis de Datos con Dart

Alumna: Cristal Ruiz Lozano #222310571
Docente: Jesús Salas Marín

20/04/2026

Índice

Introducción.....	3
Estructura del proyecto	3
Archivo JSON (datos.json).....	3
Archivo main.dart.....	4
Importación de librerías	4
Definición de la clase Registro.....	4
Función de carga de datos (cargarDatos).....	5
Función para mostrar registros (mostrarRegistros)	6
Funciones de búsqueda y filtrado	6
Funciones de estadísticas	7
Función de exportación (exportarResumen).....	8
Función del menú interactivo (menu)	9
Función principal (main).....	11
Comando de ejecución	11
Resultados	12
Opción 1 - Mostrar registros	12
Opción 2 - Buscar por nombre	12
Opción 3 - Filtrar por edad	12
Opción 4 - Filtrar por salario.....	13
Opción 5 - Ver estadísticas	14
Opción 6 - Exportar resumen	14
Opción 0 - Salir.....	14
Conclusión.....	15

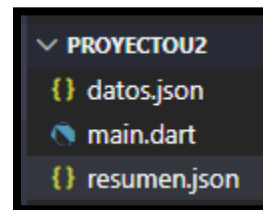
Introducción

Este proyecto de evaluación consiste en una aplicación de consola desarrollada en Dart que permite cargar, analizar y generar reportes a partir de datos almacenados en formato JSON. La aplicación implementa Null Safety, clases y objetos, estructuras de datos y un menú interactivo para facilitar la interacción con el usuario.

Estructura del proyecto

El proyecto se organiza en una estructura de archivos sencilla pero completa, que separa los datos del código fuente y permite la generación de nuevos archivos como resultado del procesamiento.

- **main.dart:** Código fuente Contiene toda la lógica de la aplicación: clase Registro, funciones de carga, búsqueda, filtrado, estadísticas, exportación y menú interactivo
- **datos.json:** Datos de entrada Archivo JSON con 20 registros de ejemplo que sirven como fuente de datos para el análisis
- **resumen.json:** Datos de salida Archivo generado automáticamente por la opción 6 del menú, contiene las estadísticas calculadas y la fecha del reporte.



Archivo JSON (datos.json)

El archivo datos.json es la fuente de datos principal de la aplicación. Contiene un arreglo de 20 objetos, donde cada objeto representa a una persona con tres campos: nombre, edad y salario.

```
[
  {"nombre": "John Smith", "edad": 28, "salario": 5500},
  {"nombre": "Emily Johnson", "edad": 34, "salario": 7200},
  {"nombre": "Michael Brown", "edad": 22, "salario": 4100},
  {"nombre": "Sarah Davis", "edad": 29, "salario": 6300},
  {"nombre": "David Wilson", "edad": 41, "salario": 8000},
  {"nombre": "Jessica Taylor", "edad": 26, "salario": 4800},
  {"nombre": "Daniel Anderson", "edad": 31, "salario": 6700},
  {"nombre": "Ashley Thomas", "edad": 27, "salario": 5200},
  {"nombre": "Matthew Jackson", "edad": 36, "salario": 7500},
  {"nombre": "Amanda White", "edad": 24, "salario": 3900},
  {"nombre": "Christopher Harris", "edad": 45, "salario": 9200},
  {"nombre": "Jennifer Martin", "edad": 33, "salario": 7100},
  {"nombre": "Joshua Thompson", "edad": 30, "salario": 6000},
  {"nombre": "Megan Garcia", "edad": 28, "salario": 5400},
  {"nombre": "Andrew Martinez", "edad": 39, "salario": 8300},
  {"nombre": "Lauren Robinson", "edad": 35, "salario": 6900},
  {"nombre": "Brandon Clark", "edad": 23, "salario": 4200},
  {"nombre": "Samantha Rodriguez", "edad": 32, "salario": 6600},
  {"nombre": "Kevin Lewis", "edad": 37, "salario": 7800},
  {"nombre": "Rachel Lee", "edad": 25, "salario": 4700}
]
```

Archivo main.dart

Importación de librerías

Las primeras líneas del código importan dos librerías fundamentales de Dart. La librería `dart:io` proporciona clases para trabajar con entradas y salidas del sistema, como la lectura de archivos (`File`), la entrada del usuario por consola (`stdin`) y la salida de datos (`stdout`). La librería `dart:convert` es necesaria para manipular datos en formato JSON, ya que incluye las funciones `jsonDecode()` (para convertir JSON a objetos Dart) y `jsonEncode()` (para convertir objetos Dart a JSON). Sin estas importaciones, el programa no podría leer el archivo `datos.json` ni generar el archivo `resumen.json`.

```
import 'dart:io';  
import 'dart:convert';
```

Definición de la clase Registro

La clase `Registro` es el modelo de datos principal de la aplicación. Cada instancia de esta clase representa a una persona con tres atributos: nombre (`String`), edad (`int`) y salario (`double`). El constructor utiliza la palabra clave `required` para asegurar que todos los parámetros sean proporcionados al crear un objeto, cumpliendo así con las normas de `Null Safety` de Dart.

El método `fromJson()` es un "factory constructor" que recibe un mapa (la representación Dart de un objeto JSON) y lo transforma en un objeto `Registro`. Utiliza el operador de coalescencia nula `??` para asignar valores por defecto ('Desconocido', 0, 0) en caso de que algún campo falte en el JSON, previniendo así errores por datos nulos.

El método `toJson()` hace la operación inversa: convierte un objeto `Registro` en un mapa que puede ser transformado a JSON para su exportación. El método `toString()` está sobrescrito con `@override` para personalizar cómo se muestra cada registro en la consola, mostrando nombre, edad y salario en un formato legible.

```

class Registro {
  String nombre;
  int edad;
  double salario;

  Registro({
    required this.nombre,
    required this.edad,
    required this.salario,
  });

  factory Registro.fromJson(Map<String, dynamic> json) {
    return Registro(
      nombre: json['nombre'] ?? 'Desconocido',
      edad: json['edad'] ?? 0,
      salario: (json['salario'] ?? 0).toDouble(),
    );
  }

  Map<String, dynamic> toJson() {
    return {
      'nombre': nombre,
      'edad': edad,
      'salario': salario,
    };
  }
}

```

Función de carga de datos (cargarDatos)

La función cargarDatos() es responsable de leer el archivo JSON y convertirlo en una lista de objetos Registro. Recibe como parámetro la ruta del archivo (por ejemplo, 'datos.json'). Dentro de un bloque try-catch para manejar errores, primero crea un objeto File con la ruta proporcionada y verifica si el archivo existe usando existsSync(). Si no existe, muestra un mensaje de error y retorna una lista vacía.

Si el archivo existe, lee su contenido como texto con readAsStringSync() y lo decodifica de JSON a una lista de objetos Dart usando jsonDecode(). Luego, utiliza el método map() para transformar cada elemento de la lista en un objeto Registro mediante el constructor Registro.fromJson(), y finalmente convierte el resultado en una lista con toList(). Si ocurre cualquier error durante este proceso (por ejemplo, JSON mal formado), el bloque catch lo captura, muestra el error y retorna una lista vacía, evitando que el programa termine inesperadamente.

```

List<Registro> cargarDatos(String ruta) {
    try {
        final file = File(ruta);

        if (!file.existsSync()) {
            print("Error: Archivo no encontrado.");
            return [];
        }

        final contenido = file.readAsStringSync();
        final List<dynamic> datos = jsonDecode(contenido);

        return datos.map((e) => Registro.fromJson(e)).toList();
    } catch (e) {
        print("Error al cargar JSON: $e");
        return [];
    }
}

```

Función para mostrar registros (mostrarRegistros)

La función `mostrarRegistros()` recibe una lista de objetos `Registro` y la muestra en la consola. Primero verifica si la lista está vacía; de ser así, muestra un mensaje informativo y sale de la función. Si la lista tiene elementos, utiliza un bucle `for` para recorrer cada registro y llamar a su método `toString()`, que ya fue sobrescrito para mostrar nombre, edad y salario en un formato legible. Esta función es utilizada por la opción 1 del menú (mostrar todos los registros) y también por las opciones de búsqueda y filtrado para mostrar los resultados.

```

void mostrarRegistros(List<Registro> lista) {
    if (lista.isEmpty) {
        print("No hay datos.");
        return;
    }

    for (var r in lista) {
        print("${r.nombre} | Edad: ${r.edad} | Salario: ${r.salario}");
    }
}

```

Funciones de búsqueda y filtrado

Estas tres funciones implementan las capacidades de búsqueda y filtrado de la aplicación:

- **buscarPorNombre():** Recibe una lista de registros y un nombre a buscar. Utiliza el método `where()` para filtrar los registros cuyo nombre contenga el texto buscado. Para hacer la búsqueda insensible a mayúsculas/minúsculas, convierte tanto el nombre del registro como el texto de búsqueda a minúsculas usando `toLowerCase()`. El método `contains()` permite encontrar coincidencias parciales (por ejemplo, buscar "john" encontrará "John Smith" y "Emily Johnson").
- **filtrarPorEdad():** Recibe una lista de registros y una edad mínima. Retorna todos los registros cuya edad sea mayor o igual al valor proporcionado.

- **filtrarPorSalario():** Recibe una lista de registros y un salario mínimo. Retorna todos los registros cuyo salario sea mayor o igual al valor proporcionado.

Las tres funciones utilizan el método `where()` que itera sobre la lista y retorna una nueva lista con los elementos que cumplen la condición, aplicando luego `toList()` para materializar el resultado.

```
List<Registro> buscarPorNombre(List<Registro> lista, String nombre) {
    return lista
        .where((r) =>
            r.nombre.toLowerCase().contains(nombre.toLowerCase()))
        .toList();
}

List<Registro> filtrarPorEdad(List<Registro> lista, int edadMin) {
    return lista.where((r) => r.edad >= edadMin).toList();
}

List<Registro> filtrarPorSalario(List<Registro> lista, double salarioMin) {
    return lista.where((r) => r.salario >= salarioMin).toList();
}
```

Funciones de estadísticas

Este conjunto de funciones calcula diversas estadísticas sobre los datos cargados:

- **totalRegistros():** Retorna la cantidad de elementos en la lista accediendo directamente a la propiedad `length`.
- **promedioSalario() y promedioEdad():** Calculan el promedio usando el método `fold()`, que acumula la suma de todos los valores (salarios o edades). Luego dividen la suma entre la cantidad de registros. Incluyen una validación inicial para evitar división por cero si la lista está vacía.
- **edadMin(), edadMax(), salarioMin(), salarioMax():** Encuentran valores extremos utilizando `map()` para extraer solo el campo deseado (edad o salario) y `reduce()` para comparar los valores y quedarse con el mínimo o máximo mediante el operador ternario `a < b ? a : b`.
- **salariosAltos() y menores30():** Utilizan `where()` para filtrar los registros que cumplen una condición específica (`salario ≥ 7000` o `edad < 30`) y luego `length` para contar cuántos cumplen.

Todas estas funciones incluyen una condición de guardia `if (lista.isEmpty) return 0;` para manejar casos extremos y evitar errores.

```
double promedioSalario(List lista) {
    if (lista.isEmpty) return 0;
    double suma = lista.fold(0, (sum, r) => sum + r.salario);
    return suma / lista.length;
}

double promedioEdad(List lista) {
    if (lista.isEmpty) return 0;
    double suma = lista.fold(0, (sum, r) => sum + r.edad);
    return suma / lista.length;
}

int edadMin(List lista) {
    if (lista.isEmpty) return 0;
    return lista.map((r) => r.edad).reduce((a, b) => a < b ? a : b);
}

int edadMax(List lista) {
    if (lista.isEmpty) return 0;
    return lista.map((r) => r.edad).reduce((a, b) => a > b ? a : b);
}
```

```
double salarioMax(List lista) {
    if (lista.isEmpty) return 0;
    return lista.map((r) => r.salario).reduce((a, b) => a > b ? a : b);
}

int salariosAltos(List lista) {
    return lista.where((r) => r.salario >= 7000).length;
}

int menores30(List lista) {
    return lista.where((r) => r.edad < 30).length;
}

int totalRegistros(List lista) {
    return lista.length;
}
```

Función de exportación (exportarResumen)

La función `exportarResumen()` genera un archivo JSON con un reporte estadístico completo de los datos procesados. Recibe la lista de registros y la ruta donde se guardará el archivo (por ejemplo, 'resumen.json'). Dentro de un bloque try-catch para manejo de errores, crea un objeto `File` con la ruta proporcionada.

Luego construye un mapa llamado `resumen` que contiene diez campos: todas las estadísticas calculadas por las funciones del fragmento anterior (total de registros, promedio de salario, edad mínima, edad máxima, promedio de edad, salario mínimo, salario máximo, cantidad de salarios altos, cantidad de menores de 30 años) y la fecha actual del sistema obtenida con `DateTime.now()`. La fecha se formatea usando `toString().split(' ')[0]` para quedarse solo con la parte de la fecha (YYYY-MM-DD).

Finalmente, convierte el mapa a JSON con `jsonEncode()` y lo escribe en el archivo usando `writeAsStringSync()`. Si todo es exitoso, muestra un mensaje de confirmación. Si ocurre algún error (por ejemplo, permisos de escritura), el bloque catch lo captura y muestra el error sin terminar el programa.


```

void exportarResumen(List<Registro> lista, String ruta) {
    try {
        final file = File(ruta);

        Map<String, dynamic> resumen = {
            'total': totalRegistros(lista),
            'promedio_salario': promedioSalario(lista),
            'edad_min': edadMin(lista),
            'edad_max': edadMax(lista),
            'promedio_edad': promedioEdad(lista),
            'salario_min': salarioMin(lista),
            'salario_max': salarioMax(lista),
            'salarios_altos': salariosAltos(lista),
            'menores_30': menores30(lista),
            'fecha_reporte': DateTime.now().toString().split(' ')[0]
        };

        file.writeAsStringSync(jsonEncode(resumen));
        print("Resumen exportado correctamente.");
    } catch (e) {
        print("Error al exportar: $e");
    }
}

```

Función del menú interactivo (menu)

La función menu() es el corazón de la interfaz de usuario. Implementa un bucle infinito while(true) que mantiene la aplicación activa hasta que el usuario elige salir. En cada iteración, muestra un menú con 7 opciones (6 funcionalidades más la opción de salir) y espera la entrada del usuario con stdin.readLineSync().

La entrada se almacena en una variable anulable (String? opcion) y se evalúa con una estructura switch. Cada caso corresponde a una opción numérica del menú:

- **Opción 1:** Llama a mostrarRegistros() para mostrar todos los registros.
- **Opción 2:** Solicita un nombre, valida que no esté vacío, y muestra los resultados de buscarPorNombre().
- **Opción 3:** Solicita una edad mínima, usa int.tryParse() para convertirla de forma segura (si la conversión falla, muestra "Edad inválida"), y muestra los resultados de filtrarPorEdad().
- **Opción 4:** Similar a la opción 3 pero con double.tryParse() para salarios.
- **Opción 5:** Muestra todas las estadísticas calculadas.
- **Opción 6:** Llama a exportarResumen() para generar el archivo JSON.
- **Opción 0:** Muestra "Saliendo..." y usa return para salir de la función (y del programa).
- **default:** Para cualquier otra entrada, muestra "Opción inválida."

Después de ejecutar la opción seleccionada (excepto la opción 0), el bucle continúa y el menú se vuelve a mostrar, permitiendo al usuario realizar múltiples operaciones sin reiniciar el programa.

```

void menu(List lista) {
    while (true) {
        print('')
        ===== MENU =====
        1. Mostrar registros
        2. Buscar por nombre
        3. Filtrar por edad
        4. Filtrar por salario
        5. Ver estadísticas
        6. Exportar resumen
        0. Salir
        '');

        String? opcion = stdin.readLineSync();

        switch (opcion) {
            case '1':
                mostrarRegistros(lista);
                break;

            case '2':
                print("Ingresa nombre:");
                String? nombre = stdin.readLineSync();

```

```

        if (nombre != null && nombre.isNotEmpty) {
            mostrarRegistros(buscarPorNombre(lista, nombre));
        } else {
            print("Entrada inválida.");
        }
        break;

        case '3':
            print("Edad mínima:");
            String? edad = stdin.readLineSync();

            int? edadInt = int.tryParse(edad ?? '');
            if (edadInt != null) {
                mostrarRegistros(filtrarPorEdad(lista, edadInt));
            } else {
                print("Edad inválida.");
            }
            break;

        case '4':
            print("Salario mínimo:");
            String? salario = stdin.readLineSync();

```

```

            double? salarioDouble = double.tryParse(salario ?? '');
            if (salarioDouble != null) {
                mostrarRegistros(
                    filtrarPorSalario(lista, salarioDouble));
            } else {
                print("Salario inválido.");
            }
            break;

        case '5':
            print("Total: ${totalRegistros(lista)}");
            print("Promedio salario: ${promedioSalario(lista)}");
            print("Edad mínima: ${edadMin(lista)}");
            print("Edad máxima: ${edadMax(lista)}");
            print("Promedio edad: ${promedioEdad(lista)}");
            break;

        case '6':
            exportarResumen(lista, 'resumen.json');
            break;

```

```

        case '6':
            exportarResumen(lista, 'resumen.json');
            break;

        case '0':
            print("Saliendo...");
            return;

        default:
            print("Opción inválida.");
        }
    }
}

```

Función principal (main)

La función main() es el punto de entrada de toda aplicación Dart. Su flujo es simple pero crucial:

1. Llama a cargarDatos('datos.json') para leer el archivo JSON y convertirlo en una lista de objetos Registro. El resultado se almacena en la variable datos.
2. Verifica si la lista está vacía usando datos.isEmpty. Si está vacía, significa que hubo un error al cargar el archivo (archivo no encontrado, JSON mal formado, etc.). En ese caso, muestra un mensaje de error y usa return para terminar el programa sin ejecutar el menú.
3. Si la lista contiene datos (es decir, la carga fue exitosa), llama a la función menu(datos) pasando la lista como argumento. A partir de este punto, el control del programa pasa al menú interactivo, que se mantendrá ejecutándose hasta que el usuario elija la opción 0 (Salir).

Esta estructura asegura que el programa solo intente mostrar el menú y procesar datos si la carga fue exitosa, evitando errores en tiempo de ejecución.

```
void main() {  
  List<Registro> datos = cargarDatos('datos.json');  
  
  if (datos.isEmpty) {  
    print("No se pudieron cargar datos.");  
    return;  
  }  
  
  menu(datos);  
}
```

Comando de ejecución

Para ejecutar la aplicación, se abre una terminal en la carpeta del proyecto y se ingresa el siguiente comando: **dart run main.dart**

Al ejecutarlo, el programa carga automáticamente el archivo **datos.json** y presenta un menú interactivo con **7 opciones** disponibles.

```
C:\Users\52871\OneDrive\Escritorio\ProyectoU2>dart run main.dart  
==== MENU ====  
1. Mostrar registros  
2. Buscar por nombre  
3. Filtrar por edad  
4. Filtrar por salario  
5. Ver estadísticas  
6. Exportar resumen  
0. Salir
```

Resultados

Opción 1 - Mostrar registros

Muestra los 20 registros del archivo JSON en formato:

```
C:\Users\52871\OneDrive\Escritorio\ProyectoU2>dart run main.dart
===== MENU =====
1. Mostrar registros
2. Buscar por nombre
3. Filtrar por edad
4. Filtrar por salario
5. Ver estadísticas
6. Exportar resumen
0. Salir

1
John Smith | Edad: 28 | Salario: 5500.0
Emily Johnson | Edad: 34 | Salario: 7200.0
Michael Brown | Edad: 22 | Salario: 4100.0
Sarah Davis | Edad: 29 | Salario: 6300.0
David Wilson | Edad: 41 | Salario: 8000.0
Jessica Taylor | Edad: 26 | Salario: 4800.0
Daniel Anderson | Edad: 31 | Salario: 6700.0
Ashley Thomas | Edad: 27 | Salario: 5200.0
Matthew Jackson | Edad: 36 | Salario: 7500.0
Amanda White | Edad: 24 | Salario: 3900.0
Christopher Harris | Edad: 45 | Salario: 9200.0
Jennifer Martin | Edad: 33 | Salario: 7100.0
Joshua Thompson | Edad: 30 | Salario: 6000.0
Megan Garcia | Edad: 28 | Salario: 5400.0
Andrew Martinez | Edad: 39 | Salario: 8300.0
Lauren Robinson | Edad: 35 | Salario: 6900.0
Brandon Clark | Edad: 23 | Salario: 4200.0
Samantha Rodriguez | Edad: 32 | Salario: 6600.0
Kevin Lewis | Edad: 37 | Salario: 7800.0
Rachel Lee | Edad: 25 | Salario: 4700.0
```

Opción 2 - Buscar por nombre

Luego se ingresa **john** y se encuentra 2 coincidencias (John Smith y Emily Johnson).

```
===== MENU =====
1. Mostrar registros
2. Buscar por nombre
3. Filtrar por edad
4. Filtrar por salario
5. Ver estadísticas
6. Exportar resumen
0. Salir

2
Ingresa nombre:
john
John Smith | Edad: 28 | Salario: 5500.0
Emily Johnson | Edad: 34 | Salario: 7200.0
```

Opción 3 - Filtrar por edad

Prueba 1 - Entrada inválida: 3 → se ingresa abc → muestra "Edad inválida."

```
===== MENU =====
1. Mostrar registros
2. Buscar por nombre
3. Filtrar por edad
4. Filtrar por salario
5. Ver estadísticas
6. Exportar resumen
0. Salir

3
Edad mínima:
abc
Edad inválida.
```

Prueba 2 - Entrada válida: 3 → ingresa 28 → muestra 14 registros con edad ≥ 28 años.

```
===== MENU =====
1. Mostrar registros
2. Buscar por nombre
3. Filtrar por edad
4. Filtrar por salario
5. Ver estadísticas
6. Exportar resumen
0. Salir

3
Edad mínima:
28
John Smith | Edad: 28 | Salario: 5500.0
Emily Johnson | Edad: 34 | Salario: 7200.0
Sarah Davis | Edad: 29 | Salario: 6300.0
David Wilson | Edad: 41 | Salario: 8000.0
Daniel Anderson | Edad: 31 | Salario: 6700.0
Matthew Jackson | Edad: 36 | Salario: 7500.0
Christopher Harris | Edad: 45 | Salario: 9200.0
Jennifer Martin | Edad: 33 | Salario: 7100.0
Joshua Thompson | Edad: 30 | Salario: 6000.0
Megan Garcia | Edad: 28 | Salario: 5400.0
Andrew Martinez | Edad: 39 | Salario: 8300.0
Lauren Robinson | Edad: 35 | Salario: 6900.0
Samantha Rodriguez | Edad: 32 | Salario: 6600.0
Kevin Lewis | Edad: 37 | Salario: 7800.0
```

Opción 4 - Filtrar por salario

Entrada: 4 → se ingresa 7000 → muestra 7 registros con salario $\geq \$7,000$.

```
===== MENU =====
1. Mostrar registros
2. Buscar por nombre
3. Filtrar por edad
4. Filtrar por salario
5. Ver estadísticas
6. Exportar resumen
0. Salir

4
Salario mínimo:
7000
Emily Johnson | Edad: 34 | Salario: 7200.0
David Wilson | Edad: 41 | Salario: 8000.0
Matthew Jackson | Edad: 36 | Salario: 7500.0
Christopher Harris | Edad: 45 | Salario: 9200.0
Jennifer Martin | Edad: 33 | Salario: 7100.0
Andrew Martinez | Edad: 39 | Salario: 8300.0
Kevin Lewis | Edad: 37 | Salario: 7800.0
```

Opción 5 - Ver estadísticas

Entrada: 5 → muestra total, promedios, edad mínima y máxima.

```
===== MENU =====
1. Mostrar registros
2. Buscar por nombre
3. Filtrar por edad
4. Filtrar por salario
5. Ver estadísticas
6. Exportar resumen
0. Salir

5
Total: 20
Promedio salario: 6270.0
Edad mínima: 22
Edad máxima: 45
Promedio edad: 31.25
```

Opción 6 - Exportar resumen

Entrada: 6 → genera el archivo **resumen.json** con todas las estadísticas.

```
===== MENU =====
1. Mostrar registros
2. Buscar por nombre
3. Filtrar por edad
4. Filtrar por salario
5. Ver estadísticas
6. Exportar resumen
0. Salir

6
Resumen exportado correctamente.
```

```
← ↻ ⓘ Archivo C:/Users/52871/OneDrive/Escritorio/ProyectoU2/resumen.json
Impresión con sangría ☒

{
  "total": 20,
  "promedio_salario": 6270,
  "edad_min": 22,
  "edad_max": 45,
  "promedio_edad": 31.25,
  "salario_min": 3900,
  "salario_max": 9200,
  "salarios_altos": 7,
  "menores_30": 9,
  "fecha_reporte": "2026-04-20"
}
```

Opción 0 - Salir

Entrada: 0 → termina la ejecución del programa.

```
===== MENU =====
1. Mostrar registros
2. Buscar por nombre
3. Filtrar por edad
4. Filtrar por salario
5. Ver estadísticas
6. Exportar resumen
0. Salir

0
Saliendo...

C:\Users\52871\OneDrive\Escritorio\ProyectoU2>
```

Conclusión

El desarrollo de este proyecto en Dart ha permitido implementar exitosamente una herramienta de análisis de datos que cumple con todos los requerimientos del proyecto. Se logró la correcta carga del archivo datos.json, la implementación de búsqueda y filtrado de registros, el cálculo de estadísticas precisas y la exportación de resultados a un nuevo archivo JSON. La aplicación aplicó adecuadamente los principios de Null Safety mediante el uso de required, operador ?? y tryParse(), además de estructuras de datos como fold(), map() y reduce() para el procesamiento eficiente de la información. El menú interactivo funciona correctamente validando entradas y manejando errores, lo que demuestra el dominio de Dart para el desarrollo de aplicaciones de consola robustas y funcionales.